

AUTOMATIC WATER LEVEL CONTROL SYSTEM

EMBEDDED MICROCONTROLLERS

KELLY ATIENO DIANGA

Contents

PROPOSED CASE	3
CASE DESCRIPTION	3
ADVANTAGES OFFERED BY THE SYSTEM	3
DISADVANTAGES OFFERED BY THE SYSTEM	3
HOW IT WORKS	3
Using Ultrasonic Sensor to detect water levels	3
Using microcontroller to control water pump	3
Calculating the Pump rate.....	4
Using Terminate to display water level,When pump is on or off and pump rate.....	4
Using two port pins to represent water level going above/below the set marks.....	4
ILLUSTRATION OF PROPOSED CASE	4
I/O TABLE	5
FLOWCHART FOR PROPOSED CASE	6
FULL SOURCE CODE	6
IMPROVEMENTS TO BE MADE TO THE PROGRAM	9
1. Creating a delay function	9
2. Clearing the display screen after problem is tackled	10
3. Calculate the amount of time it takes to drain the tank	10
FLOWCHART FOR IMPROVED PROGRAM	10
WEB LINK FOR DEMONSTRATION VIDEO	10
REFERENCES	10

PROPOSED CASE

Using a microcontroller GPIO, LCD Screen, FTM timer and an Ultrasonic sensor to design a system that automatically controls the water level in a tank by turning the pump on and off.

CASE DESCRIPTION

Attached to a water tank is a pair of ultrasonic sensors, water pump and a tap. The ultrasonic sensor is used to control the water level by triggering the microcontroller to turn the pump on and off. The water pump is turned on when the water level falls below the too low mark and stays on until it rises above the too high mark.

ADVANTAGES OFFERED BY THE SYSTEM

Some of the advantages offered by the system include;

1. Save electrical sources by minimising the rate at which the pump is turned on and off
2. Saves water resources by preventing water from overflowing when the pump is turned on hence preventing water wastage.
3. Can easily be installed in different homes, hospitals, hotels, e.t.c
4. System can maintain the water level thus preventing disruption of daily activities requiring water usage for example taking a shower.
5. Maintenance cost is very low and sensors can be free from corrosion
6. Amount of power consumed by the system is very low
7. System is compact hence making it easy to move around
8. Automatic thus requires little to no manual monitoring

DISADVANTAGES OFFERED BY THE SYSTEM

Just like every system, it also has a few downsides. These are:

1. Needs to be replaced frequently, roughly every 3 years
2. Contaminates water when the metal parts rust and deteriorate
3. Comes with no warranty/Guarantee

HOW IT WORKS

Using Ultrasonic Sensor to detect water levels

The ultrasonic sensor is attached to the lower water mark and high water mark inside the tank so as to detect its water levels: whether too high or too low.

Using microcontroller to control water pump

The ultrasonic sensors are interfaced with the microcontroller. FTM Capture input is then used to control the pump. When the water level rises above the high level mark, the sensor sends a signal to the FTM module and it turns off the pump. When the water level goes below the low level mark, the signal sent by the sensor triggers the FTM module to turn on the water pump.

Calculating the Pump rate

I will use the FTM input Capture to record the time the pump is turned on and off so as to calculate the time it takes to fill the tank.

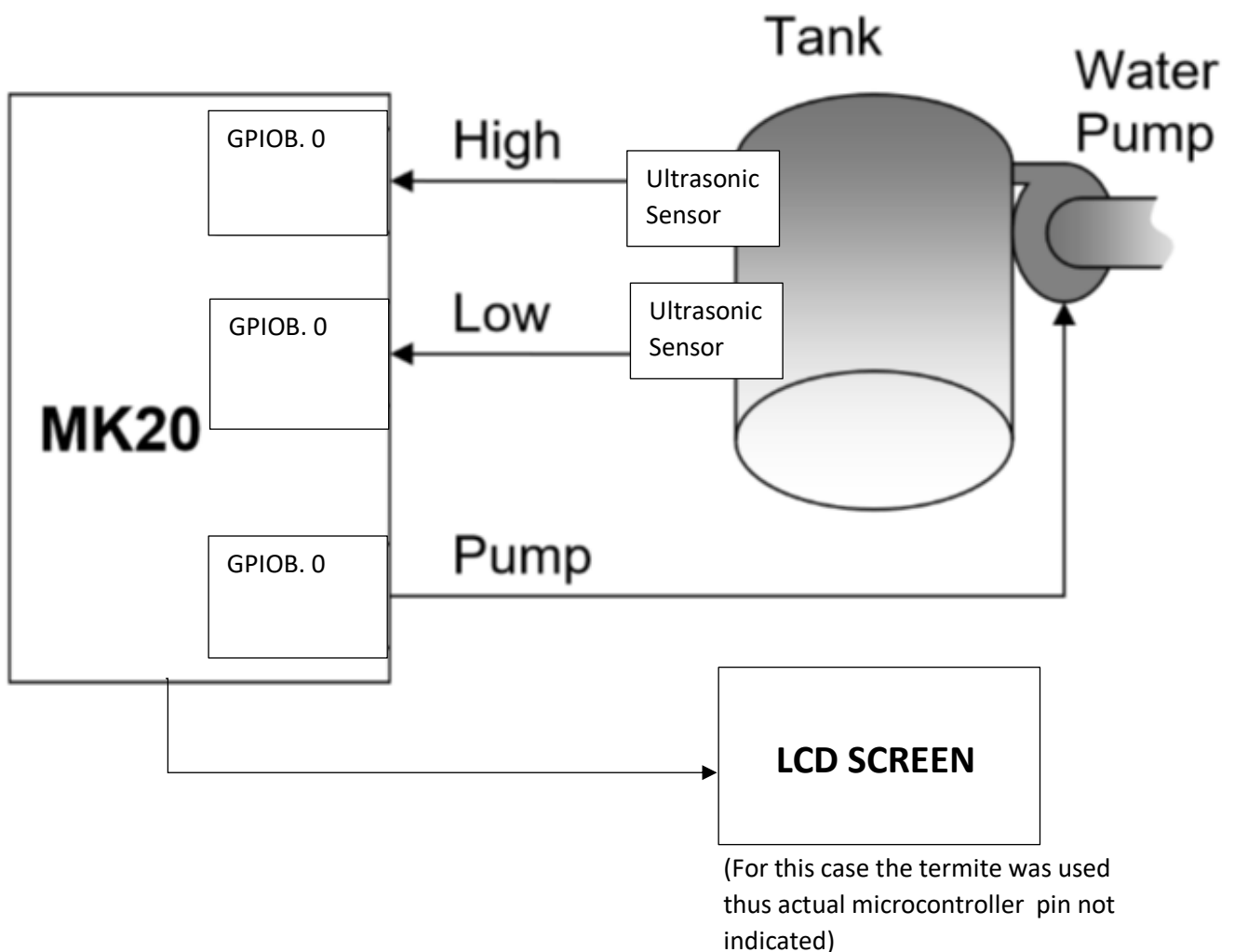
Using Termite to display water level, When pump is on or off and pump rate.

The LCD screen was to be used for displaying warnings whenever the water level went below the marks, show when the pump is turned on or off and display the time it takes to fill the tank. However, a specific hardware such as the LCD screen could be used for this purpose.

Using two port pins to represent water level going above/below the set marks.

Two Port pins will be used to represent the water level and show the system reacting to water level rising above high level mark and going below too low mark.

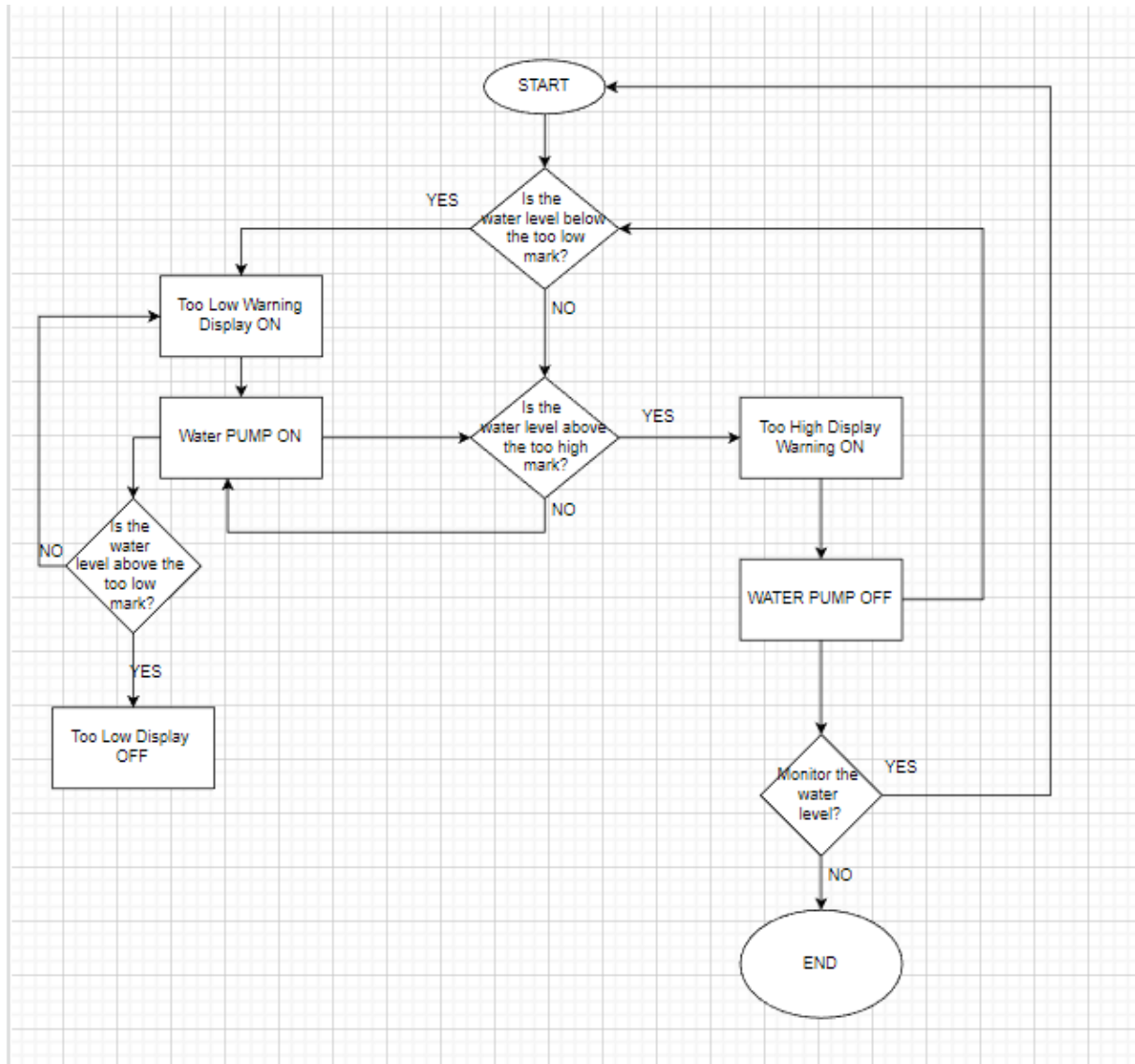
ILLUSTRATION OF PROPOSED CASE



I/O TABLE

NAME	BIT	TYPE	FUNCTION
Water Pump	8	Output	Turned on when water fall below too low mark and turned off when water level rises above too high mark.
Too high water level	0	Input	Represents water level falling below the too low water mark
Too low water level	1	Input	Represents water level rising above the too high water mark
Too low sensor	0		FTM1 Channel 0 used to 1. confirm water level as too low 2. turn on water pump 3. Record time water pump is turned on
Too high Sensor	1		FTM1 Channel 1 used to 1. confirm water level as too high 2. turn off water pump 3. Record time water pump is turned off

FLOWCHART FOR PROPOSED CASE



FULL SOURCE CODE

```
/*
 *
 * Using FTM Input capture to Control water level in a tank and record the
 * time it takes to fill it
 *
 */

/*Include header files*/
#include <stdio.h>
#include "derivative.h"
#include "clock.h"
#include "utilities.h"
typedef enum {false=0, true=!false} bool;
```

```

// System clock frequency
#define SYSTEM_CLOCK_FREQUENCY (48000000UL)

// Choice of prescale value (FTM1_SC.PS)
#define FTM1_PRESCALE_VALUE      (0)
#define FTM1_PRESCALE            (1<<FTM1_PRESCALE_VALUE)
#define FTM1_CLK_FREQUENCY       (SYSTEM_CLOCK_FREQUENCY/FTM1_PRESCALE)
#define TICKS_IN_ONE_MICROSECOND (FTM1_CLK_FREQUENCY/1000000)

// The maximum period that can be measured (in us)
#define MAX_PERIOD (65536/TICKS_IN_ONE_MICROSECOND)

#if (MAX_PERIOD<1000)
#error "MAX_PERIOD is too small"
#endif

//Represents the Water Pump
#define LED_MASK (1<<8)

/*Initialise FTM1
*
* Configures:
*   Enables FTM1 clock
*   Sets FTM1 CNTIN & MOD values
*   Enables FTM1 interrupts
*/
void initialiseFTM1(void) {

    // Enable clock to FTM
    SIM_SCGC6 |= SIM_SCGC6_FTM1_MASK;

    // Common registers
    FTM1_SC      = FTM_SC_CLKS(0);
    FTM1_CNTIN   = 0;
    FTM1_CNT     = 0;
    FTM1_MOD     = 0xFFFF;
    FTM1_SC      = FTM_SC_CLKS(1) | FTM_SC_PS(FTM1_PRESCALE_VALUE);

    // Enable FTM1 interrupts in NVIC
    NVIC_EnableIrq(INT_FTM1);
}

// The following are made global to allow access from ISR
//Use to calculate time it takes to fill tank
static volatile uint16_t firstMeasurement;
static volatile bool    firstMeasurementDone;
static volatile uint16_t secondMeasurement;
static volatile bool    secondMeasurementDone;

//Use to turn the water pump on and off
static volatile bool    PumpON;
static volatile bool    PumpOFF;

/*!FTM1 Interrupt Handler
*
* It:
* 1.Turns the water pump on and Off
* 2.Takes the time measurements when the water pump is turned on and off

```

```

* 3.Displays warning whenever the water level goes below or above the too
low/too high marks
* 4.Displays message when water pump is on or off
*
*/
void FTM1_IRQHandler(void) {

    //Check if water level is below too low mark
    if ((FTM1_C0SC&FTM_CnSC_CHF_MASK) != 0) {
        // Clear the interrupt request from FTM1.Ch0
        FTM1_C0SC = FTM_CnSC_CHIE_MASK|FTM_CnSC_ELSB_MASK;

        // Display warning
        printf("Water Level TOO LOW!\n");

        // Turn on water pump
        GPIOC_PSOR = LED_MASK;

        // Display message that water pump is turned on
        printf("Water Pump ON \n\n");
        PumpON =true;

        //Record time when water pump is turned on
        firstMeasurement      = FTM1_C0V;
        firstMeasurementDone = true;
    }

    //Check if water level is above high mark
    if ((FTM1_C1SC&FTM_CnSC_CHF_MASK) != 0) {
        // Clear the interrupt request from FTM1.Ch1
        FTM1_C1SC = FTM_CnSC_CHIE_MASK|FTM_CnSC_ELSB_MASK;

        // Ignore transitions until 1st has occurred
        if (firstMeasurementDone) {

            // Display warning
            printf("Water Level TOO HIGH! \n");

            // Turn off water water pump
            GPIOC_PCOR = LED_MASK;

            // Display message that water pump is turned off
            printf("Water Pump OFF \n\n");
            PumpOFF =true;

            //Record time when water pump is turned off
            secondMeasurement      = FTM1_C1V;
            secondMeasurementDone = true;

            // Disable both channels as measurement complete
            FTM1_C0SC = 0;
            FTM1_C1SC = 0;
        }
    }

    if ((FTM1_STATUS & ~(FTM_STATUS_CH0F_MASK|FTM_STATUS_CH1F_MASK)) != 0) {
        // Unexpected interrupt  die here for debug
        __asm("bkpt");
    }
}

```

```

/*
 * Do a measurement of the time between the two input capture events
 *
 * @return period in microseconds
 *
 * @note The period measurement is limited to MAX_PERIOD us
 */
int waterLevelControl(void) {
    //initialise port clocks for port B and C
    SIM_SCGC5 |= SIM_SCGC5_PORTB_MASK;
    SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK;

    // Configure pins: PTB.0,1(Fn 3) =active low for FTM1_CH0,1
    PORTB_PCR0 = PORT_PCR_MUX(3)|PORT_PCR_PE_MASK|PORT_PCR_PS_MASK;
    PORTB_PCR1 = PORT_PCR_MUX(3)|PORT_PCR_PE_MASK|PORT_PCR_PS_MASK;

    //Make Port C pin 8 simple port pin
    PORTC_PCR8 = PORT_PCR_MUX(1);

    //Make port C pin 8 an output
    GPIOC_PDDR = LED_MASK;

    PumpON = false;
    PumpOFF = false;

    firstMeasurementDone = false;
    secondMeasurementDone = false;

    // Input capture on falling edges
    FTM1_C0SC = FTM_CnSC_CHIE_MASK|FTM_CnSC_ELSB_MASK;
    FTM1_C1SC = FTM_CnSC_CHIE_MASK|FTM_CnSC_ELSB_MASK;

    while (!secondMeasurementDone) {
        __asm("wfi");
    }
    return (secondMeasurement-firstMeasurement)/TICKS_IN_ONE_MICROSECOND;
}

int main(void) {
    clock_initialise();
    initialiseFTM1();

    for(;;) {
        printf("Pump Rate = %d uS\n\n", waterLevelControl());
    }
}

```

IMPROVEMENTS TO BE MADE TO THE PROGRAM

1. Creating a delay function

I plan on creating a delay function and use it between displaying a warning when water level passes he marks and when the pump is turned on. This is to simulat real life scenerio where there is delay between problem and reaction to tackle the problem. This is to also allow for warnings to be read before the notification for the action to tackle the problem is displayed.

2. Clearing the display screen after problem is tackled

Another improvement I plan on making is letting the warning be displayed for the period in which water level passing the marks is detected. The warnings are then cleared when the pump is turned on or off and only display a message of whether the pump is on or off.

3. Calculate the amount of time it takes to drain the tank

I also adding further calculations to find the period it takes to drain the tank. This is so as to enable the user to tell whether he/she uses a lot of water for a short period or vice versa. They can then regulate their water usage.

WEB LINK FOR DEMONSTRATION VIDEO

<https://youtu.be/bIUOGs33dQQ>

REFERENCES

1. Automatic Water Level Indicator and Pump Controller Using Arduino, Ettron, 2020
<https://youtu.be/22U8yPX38A>
2. [12 Ultrasonic Sensor Applications | MaxBotix Inc.](#)
3. [15 Applications for Ultrasonic Sensors | Migatron Corp.](#)
4. [Automatic Water Level Controller Working Principle \(W/ Pics & Diagrams\) - Water Level Controls](#)
5. Swinburne University of technology, Embedded Microcontrollers, Tutorial 3 and Tutorial 7, 2022